

CityScan

מדריך ארכיטקטורה, זרימה ושיקולי תכנון

FastAPI + Gemini

MongoDB

TypeScript

Node.js + Express 5

React 19 + Vite

Leaflet

אפליקציית Web לדיווח על מפגעים עירוניים

ארכיטקטורת שכבות בסטנדרט של פרויקט SmartCollect

אוגוסט 2026

תוכן עניינים

1. מה זה CityScan — תקציר · המשפט שאומרים ראשון
2. מה יש בפרויקט — שלושה שירותים
3. הכלים ולמה נבחרו
4. הארכיטקטורה — שני צירים
5. צינור ה-Middleware
6. הזרימה המלאה — מסלול של דיווח אחד
7. כל תיקייה, מה היא עושה
8. שלוש השכבות שהכי מבלבלות — data, boundaries, converters
9. מה שונה מ-SmartCollect ולמה
10. השינויים — לפני ואחרי
11. שלושת הבאגים שנמצאו ותוקנו
12. בדיקות
13. שיקולי עלות (FinOps)
14. מה הלאה — שלבים 2 ו-3
15. איך להסביר את זה לאנשים

1. מה זה CityScan

אפליקציית Web שבה תושב מדווח על מפגע עירוני — בור בכביש, פנס שבור, פסולת, הצפה — על ידי לחיצה על מפה, צילום תמונה ובחירת סוג. הדיווחים מוצגים לכולם על המפה, ולעירייה יש פאנל ניהול לעדכון סטטוס.

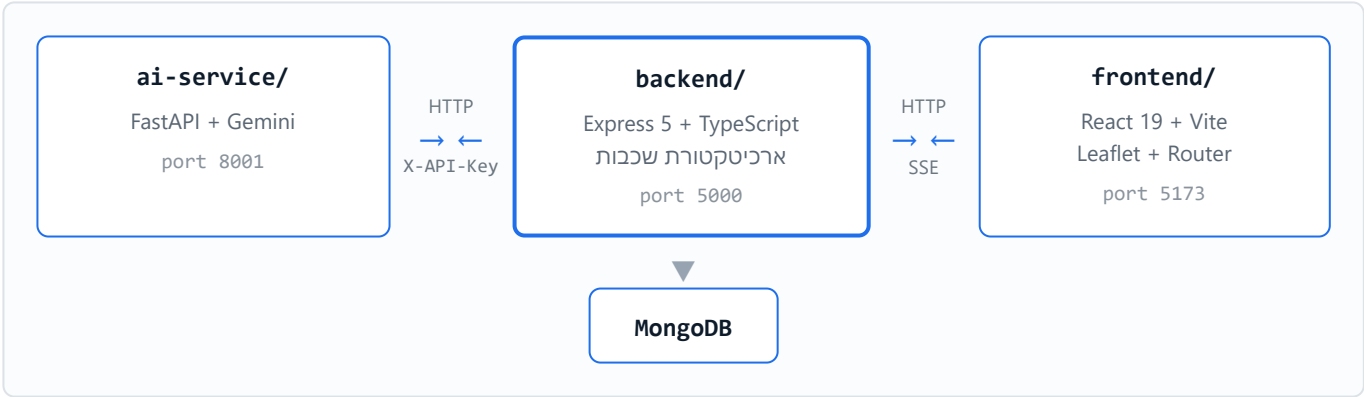
המשפט שאומרים ראשון

Elevator pitch

"CityScan היא מערכת דיווח מפגעים עירוניים בארכיטקטורת שלושה שירותים: Frontend ב-API, React ב-Node/TypeScript בארכיטקטורת שכבות קפדנית, ושירות AI נפרד בפיתוח. הדבר המעניין טכנית הוא **מניעת דיווחים כפולים בשתי רמות** — שאילתה גאוגרפית על אינדקס `2dsphere` תופסת את המקרה הוודאי, ו-LLM מכריע במקרים המעורפלים. והכל בעלות של \$5 לחודש."

יכולת	מימוש
דיווח מפגע עם מיקום מדויק	לחיצה על מפת Leaflet, שמירת <code>latitude</code> / <code>longitude</code> + GeoJSON
מניעת כפילויות ברדיוס 50 מ'	<code>\$geoWithin</code> + <code>\$centerSphere</code> על אינדקס <code>2dsphere</code>
הכרעת כפילות מעורפלת	Google Gemini דרך שירות FastAPI נפרד
תיאור תמונה אוטומטי	Gemini Vision, מאחורי פרוקסי בשרת
אימות והרשאות	JWT + bcrypt + הפרדת תפקידים <code>admin</code> / <code>user</code>
עדכון מפה בזמן אמת	Server-Sent Events — ללא ספריות חיצוניות
אחסון תמונות	דחיסה ל-WebP, ניקוי EXIF, דיסק מקומי
פאנל ניהול	סינון, חיפוש, עדכון סטטוס
מצב דמו למגייסים	כניסת אורח לקריאה בלבד

2. מה יש בפרויקט — שלושה שירותים



שירות	גודל	מה בפנים
backend/	~2,400 שורות	Node.js, Express 5, TypeScript (ESM), Mongoose. 11 שכבות. 35 בדיקות אינטגרציה.
frontend/	2,725 שורות	React 19, Vite 7, react-leaflet, Bootstrap, React Router. חמישה מסכים: מפה, התחברות, הרשמה, ההיסטוריה שלי, ניהול.
ai-service/	Python	FastAPI + Google Gemini. שני נתיבים מוגנים ב- <code>X-API-Key</code> : <code>/analyze</code> ו- <code>/check-duplicate</code> .

למה שירות AI נפרד ולא בתוך ה-Backend

שלוש סיבות שכדאי לומר: המערכת האקולוגית של ML היא פייתון; זה מבודד את התלות — אם שירות ה-AI נופל, המערכת ממשיכה לעבוד; וזה מאפשר להחליף ספק LLM בלי לגעת בלוגיקה העסקית.

3. הכלים ולמה נבחרו

כלי	תפקיד	למה הוא ולא אחר
TypeScript	שפה	מופעל עם <code>strict</code> , <code>exactOptionalPropertyTypes</code> , <code>noUncheckedIndexedAccess</code> — ההגדרות הקפדניות, לא ברירת המחדל
Express 5	שרת HTTP	מינימלי; השכבות נבנו ידנית ולכן הן מפורשות ולא מוסתרות ע"י framework
MongoDB + Mongoose	מסד נתונים	השאליתה המרכזית גאוגרפית. אינדקס <code>2dsphere</code> נותן <code>\$geoWithin</code> נייטיב. Atlas M0 חינמי ולא נכבה
Zod	ולידציה	<code>z.infer</code> גוזר את הטיפוס מהסכמה — הוולידציה והטיפוס לא יכולים להתפצל
JWT + bcrypt	אימות	Stateless. סיסמאות מגובבות ב-10 סבבים
sharp	עיבוד תמונה	הספרייה המהירה ביותר ב-Node. WebP + הסרת EXIF
helmet · rate-limit · cors	אבטחה	כותרות, הגבלת קצב, בקרת מקורות לפי פרופיל
SSE	זמן אמת	קל בהרבה מ-socket.io. חד-כיווני מספיק כאן
Vitest + mongodb-memory-server	בדיקות	DB אמיתי חד-פעמי לכל ריצה, בלי צורך ב-Docker
Leaflet + OpenStreetMap	מפה	\$0, בלי מפתח API ובלי כרטיס אשראי

שאלה שישאלו: "למה לא Google Maps?"

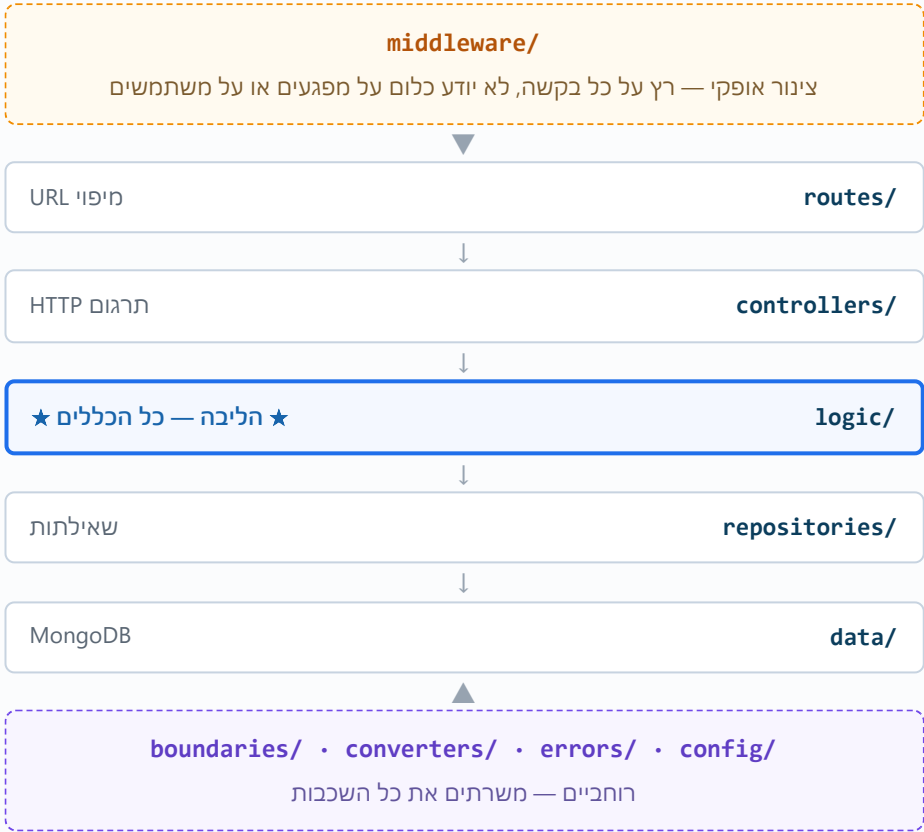
"בדקתי. מאז מרץ 2025 גוגל ביטלה את הקרדיט של \$200 והחליפה אותו במכסות לכל 10,000 — SKU טעינות מפה בחודש חינם ואז כ-\$7 לכל 1,000, וחובה חשבון חיוב עם כרטיס אשראי. אפשר להגביל בעזרת Quota יומי, שזו תקרה קשיחה אמיתית — לא Budget Alerts, שהם רק התראה ולא מגבילים שימוש. אבל כשהמכסה נגמרת המפה מתה עד חצות. לפרויקט בעלות \$5 לחודש זה לא השתלם."

שאלה שישאלו: "למה MongoDB ולא PostgreSQL?"

"השאליתה המרכזית שלי גאוגרפית, ו-Mongo נותן אותה נייטיב. PostGIS חזק יותר, אבל על VPS של \$5 שמריץ גם n8n אין זיכרון ל-Postgres מקומי, והחלופות החינמיות המנוהלות נרדמות. Atlas M0 לא נרדם. **והנקודה החשובה** — הסטנדרט ההנדסי הוא ההפרדה לשכבות, לא מנוע האחסון. בפרויקט הקודם שלי החלפתי MongoDB ב-PostgreSQL באמצע, ורק שתי שכבות השתנו. זה בדיוק המבחן שהארכיטקטורה הזו נועדה לעבור."

4. הארכיטקטורה — שני צירים

זו הנקודה שהכי חשוב להעביר נכון. יש **מחסנית אנכית** של שכבות, ויש **צינור אופקי** של `middleware` שרץ לפניה. הם לא אותו דבר.



האמצע הוא **מחסנית אנכית**: כל שכבה מדברת רק עם זו שמתחתיה. למעלה ולמטה — **צירים אופקיים** שחוצים את כולן.

חוק הזהב

חץ אחד למטה

`data/` לא יודע שקיים `logic/`. `logic/` לא יודע שקיים `HTTP`. `controllers/` מכיר רק **ממשקים** מ-`logic/`, לא מימושים.

התוצאה המעשית: אפשר לבדוק את הלוגיקה העסקית בלי להרים שרת, ואפשר להחליף את Groq ב-Gemini בלי לגעת בה.

ממשק מול מימוש

לכל שירות יש שני קבצים, וזה מבלבל בהתחלה:

קובץ	מה יש בו
logic/ai.service.ts	מה אפשר לעשות — רשימת פונקציות בלי גוף
logic/impl/gemini.ai.service.impl.ts	איך — הקוד שבאמת רץ

hazards.service.impl.ts מכיר רק את הראשון. לכן החלפת ספק ה-LLM היא קובץ חדש ושורה אחת ב-container.ts.

5. צינור ה-Middleware

ב-Spring Boot רוב הדברים האלה הם אנוטציות שה-framework מספק. ב-Express הם קוד מפורש שכתבתי — וזה בעצם יתרון הסברתי: רואים בדיוק מה קורה.

ב-CityScan (קוד)	ב-Spring Boot (אנוטציה)
errorHandler.ts	@ControllerAdvice
requireRole.ts	@PreAuthorize
validate.ts	jakarta.validation + @Valid
auth.middleware.ts	Spring Security Filter Chain
rateLimiter.ts	— (אין מקבילה מובנית)

למה זה לא "שכבה"
 שכבה עוסקת בדומיין — מפגעים, משתמשים. Middleware הוא חוצה-דומיין: אותו קוד אימות רץ על מפגעים, על משתמשים ועל לוגים, ולא יודע במה הוא מטפל. לכן הוא ציר אחר.

6. הזרימה המלאה — מסלול של דיווח אחד

מה שקורה כששתמש לוחץ "שלח דיווח", מקצה לקצה:

1 **cors** מקור מותר? בפרודקשן חובה `FRONTEND_URL`

2 **helmet** כותרות אבטחה

3 **express.json** פענוח גוף, מקסימום 8mb

4 **rateLimiter** לא חרגת ממכסה? (נתיב ה-SSE פטור)

5 **optionalAuth** יש טוקן? → `req.user`

6 **demoRestrict** אדמין-דמו? → חסום כתיבה

— עד כאן גלובלי על כל בקשה. מכאן לפי נתיב —

7 **authMiddleware** חייב טוקן, אחרת 401

8 **validate(schema)** Zod. נכשל → זורק `BadRequestException`

== נכנסים לשכבות ==

9 **HazardController.create** 3 שורות: קורא, מאציל, שולח

10 **HazardsServiceImpl.create** ★ כל הכללים ★

← `repository.findNearbyUnresolved()` — `$geoWithin`, רדיוס 50 מ'

← אותו סוג ברדיוס? זורק `ConflictException` (409)

← סוג אחר? `ai.checkDuplicate()` — נופל בחן אם השירות מת

← `photos.saveMany()` — WebP, ניקוי URL, EXIF בלבד

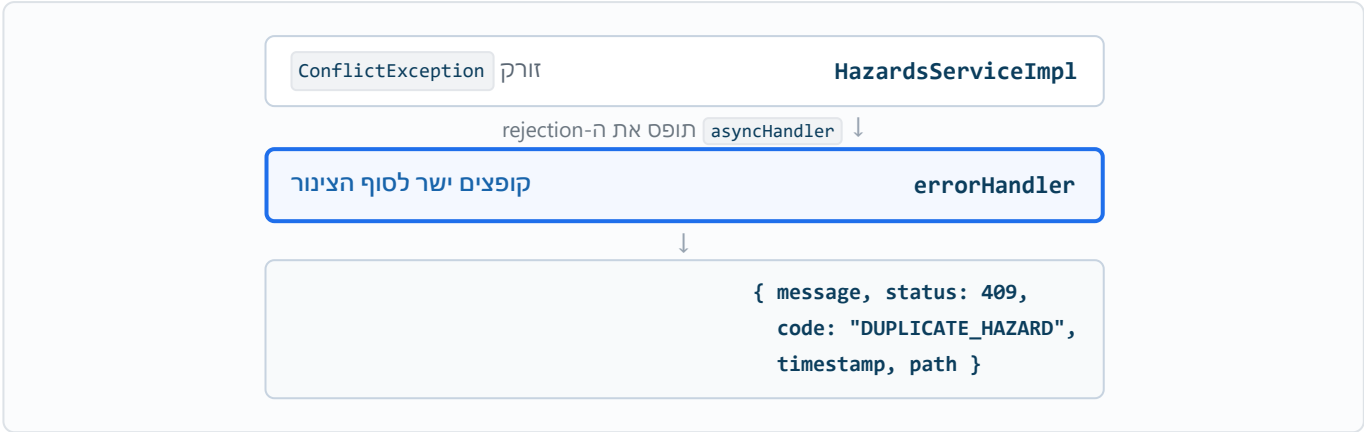
← `repository.create()` → `data/` → MongoDB

← `events.emit()` — SSE לכל הדפדפנים המחוברים

11 **converter.toBoundary()** JSON → entity נקי

12 **res.status(201).json()** התשובה ללקוח

ומה קורה כשמשהו נכשל



העיקרון

אף מקום בפרויקט חוץ מ- `errorHandler.ts` לא כותב תשובת שגיאה. השירות זורק חריגה טיפוסית שנושאת את קוד ה-HTTP שלה; הוא לא בוחר סטטוס ולא בונה JSON.

מניעת כפילויות — שתי רמות

זה הפיצ'ר שהכי שווה להסביר לעומק:

1. **רמה דטרמיניסטית:** מפגע לא פתור מאותו סוג בתוך 50 מ' הוא כפילות. בלי AI, בלי השהיה, עובד אופליין.
2. **רמה מעורפלת:** רק כשיש מפגעים בסביבה מסוג אחר. "פסולת" ו"הצפה" במרחק חמישה מטרים יכולים להיות אותה בעיה פיזית — וזו שאלה שדורשת שיפוט. היא הולכת ל-LLM.

Fail-open — החלטת תכנון מודעת

אם שירות ה-AI לא זמין, הדיווח **עובר**. תושב חייב תמיד להיות מסוגל לדווח על מפגע, גם כשתלות אופציונלית נופלת. זו גם הסיבה שבדיקת הכפילויות לא הועברה ל-n8n — היא על הנתיב הקריטי, המשתמש ממתין לתשובה, וזו לוגיקה עסקית ליבתית שצריכה להיות ב-git ומכוסה בבדיקות.

7. כל תיקייה, מה היא עושה

תיקייה	תפקיד	מותר לה להכיר
middleware/	חוצה-הכל: אבטחה, אימות, ולידציה, שגיאות	logic/ (ממשקים), errors/
routes/	URL → controller, ושרשור השומרים	controllers/, boundaries/
controllers/	תרגום HTTP בלבד. בלי try/catch	logic/ — ממשקים בלבד
logic/	מה אפשר לעשות — ממשקים בלי גוף	boundaries/
logic/impl/	איך — כל הלוגיקה העסקית	errors/, converters/, repositories/
repositories/	שאלתות Mongo. הגאוגרפיה חיה כאן	data/
data/	סכמות Mongoose + enums	כלום
boundaries/	חוצה ה-JSON + סכמות Zod, באותו קובץ	data/enums
converters/	entity ⇌ boundary. מוריד סיסמאות	שניהם
errors/	חריגות שנושאות קוד HTTP	כלום
config/	פרופילים, env מאומת, חיבור DB	כלום
container.ts	מחבר הכל — ההזרקה של Spring, ידנית	הכל

8. שלוש השכבות שהכי מבלבלות

אותו מפגע, בשלוש צורות שונות:

boundaries/

איך זה נשלח — JSON ללקוח

`_id: "64f..."`

`latitude: 32.08`

— לא קיים —

`reportedBy: { _id, email }`

`createdAt: "2026-08-07T..."`

— לא קיים —

data/

איך זה מאוחסן — MongoDB

`_id: ObjectId("64f...")`

`latitude: 32.08`

`location: { type: "Point",
coordinates: [...] }`

`reportedBy: ObjectId("507...")`

`createdAt: Date`

`__v: 0`

←
/converters
המתרגם

אידום = פנימי, לא עובר ללקוח · **ירוק** = הומר לצורה שהלקוח מצפה לה

data/ — הצורה שנשמרת

סכמות Mongoose. השדה `location` קיים רק בשביל האינדקס `2dsphere` — הוא כפילות של `latitude` / `longitude`, ו-hook שומר אותו מסונכרן. הלקוח לא רואה אותו לעולם. גם ה-`enums` יושבים כאן, ומהם קראות גם סכמת Mongoose וגם סכמת Zod — סוג מפגע חדש מתווסף במקום אחד.

boundaries/ — החוזה כלפי חוץ

שני דברים באותו קובץ: טיפוס התשובה, שחייב להיות זהה שדה-בשדה ל-`interface Hazard` בצד הלקוח; וסכמת Zod שבודקת את הבקשה.

```
export const createHazardSchema = z.object({  
  type: z.enum(HAZARD_TYPES),  
  latitude, // למספר, בין 90- ל-90  
  longitude,  
  hazardPhotos: z.array(photoPayload).max(10).optional(),  
});  
  
export type CreateHazardBoundary = z.infer<typeof createHazardSchema>;
```

השורה האחרונה היא הרעיון: **הטיפוס נגזר מהסכמה**, ולכן אי אפשר שהוולידציה תבדוק דבר אחד וה-`TypeScript` יאמין לדבר אחר.

converters/ — המתרגם

הקובץ הכי חשוב ביטחונית הוא `user.converter.ts`, כי הוא בונה את התשובה שדה-בשדה — ול-`UserBoundary` בכלל אין שדה `password`, אז הסיסמה פיזית לא יכולה לדלוף.

הקוד הישן (3 מופעים)	הקוד החדש
<pre data-bbox="1125 230 1460 302">const o = user.toObject(); delete o.password;</pre> דולף כברירת מחדל — שדה חדש בסכמה נשלח ללקוח עד שמישהו יזכור למחוק	<pre data-bbox="352 230 767 336">toBoundary(entity) { return { _id, email, role, ... } }</pre> חוסם כברירת מחדל — שדה חדש נשלח רק אם כתבת אותו במפורש

9. מה שונה מ-SmartCollect ולמה

המבנה מועתק אחד-לאחד מהפרויקט הקודם, כולל מוסכמות השמות ואפילו האי-סימטריה שבהן: שירותים ברבים (`HazardsService`), קונטרולים וישויות ביחיד (`HazardController` , `HazardEntity`).

מה נלקח

CityScan (Express)	SmartCollect (Spring Boot)
זהה	<code>controllers/</code> רזים, בלי <code>try/catch</code>
זהה	<code>logic/</code> + <code>logic/impl/</code>
מחלקות מעל Mongoose	<code>repositories/</code> (Spring Data)
<code>data/</code> (Mongoose Schema)	<code>data/</code> (<code>@Entity</code>)
זהה — פלוס סכמת Zod	<code>boundaries/</code> (DTOs)
זהה	<code>converters/</code>
<code>errors/</code> עם <code>AppException.status</code>	<code>errors/</code> עם <code>@ResponseStatus</code>
<code>NODE_ENV</code> + <code>.env.{profile}</code>	<code>@Profile</code> + <code>application-{profile}.properties</code>
<code>mongodb-memory-server</code>	DB — Testcontainers נקי לכל ריצה

מה תוקן במכוון

המדריך של SmartCollect מפרט בעצמו את חולשותיו. אלה תוקנו כאן במקום להיות מועתקות — **וזו נקודה חזקה בראיון**: לא רק שחזרת סטנדרט, זיהית איפה הוא נשבר.

CityScan-ב	חולשה ב-SmartCollect
10 bcrypt סבבים	סיסמאות בטקסט גלוי
JWT ב- <code>Authorization: Bearer</code>	סיסמה כפרמטר ב-URL ("נכנסים ללוגים")
<code>requireRole</code> כ- <code>middleware</code> על הנתיב	קריאות <code>require...</code> ידניות בכל מתודה
handler מרכזי עם חוזה מפורש	אין <code>@ControllerAdvice</code> , וגוף השגיאה לא מוגדר — ההודעות לא מגיעות ללקוח
קבצי פרופיל אמיתיים	מעבר בין סביבות ע"י הסרת הערות מקוד
<code>image</code> , <code>volume</code> מקובע, <code>healthcheck</code>	<code>compose.yaml</code> בלי <code>volume</code> — <code>down</code> מוחק את ה-DB

Docker — הבדל תפקידי חשוב

ב-SmartCollect אין `Dockerfile` בכלל: Docker היה רק נוחות פיתוח שהחזיקה את ה-PostgreSQL, והאפליקציה תמיד רצה על המחשב. ספרינג אפילו הרים את הקונטיינר לבד — ולכן *חובה* היה להפעיל Docker Desktop לפני ההרצה.

ב-CityScan Docker הוא **אופציונלי בפיתוח** (אפשר להצביע ל-Atlas ולדלג עליו), ובשלב 3 הוא הופך לסביבת **הייצור** עצמה.

10. השינויים — לפני ואחרי

היבט	לפני	אחרי
שכבות	תיקיות קיימות, אבל <code>controllers/hazards.ts</code> = 500 שורות עם הכל	4 שכבות אמיתיות; קונטרולר = 3 שורות לכל handler
טעינת env	לא נטענת בכלל (ראה סעיף 11)	<code>config/env.ts</code> מאומת ב-Zod
שגיאות	handler גלובלי שהוא קוד מת	מקום אחד שכותב שגיאה, חוזה מוגדר
ולידציה	<code>validate.ts</code> = <code>next()</code> ריק	Zod על כל נתיב
הרשאות	4 בלוקים משוכפלים בתוך הקונטרולר	<code>requireRole()</code> על הנתיב
תמונות	base64 בתוך המסמך, עד $1.5\text{MB} \times 10$	WebP על דיסק, URL ~ 150KB בלבד ב-DB
עדכון מפה	Polling כל 8 שניות, 500 מפגעים	SSE — דחיפה כשמשהו קורה
בדיקות	אין	35 בדיקות אינטגרציה
פגיעויות	7 (אחת קריטית)	0

המספר שכדאי לזכור

הפוליטיקה כל 8 שניות ייצרה ~450 בקשות בשעה לכל לקוח פתוח, כל אחת מחזירה עד 500 מפגעים עם תמונות base64. המעבר ל-SSE הוא חיבור אחד שקט. זה שיפור שהוא גם ביצועים, גם עלות רוחב-פס וגם סוללה בנייד.

11. שלושת הבאגים שנמצאו ותוקנו

אלה החומר החזק ביותר לראיון, כי הם מדגימים אבחון ולא רק כתיבה.

באג 1 — משתני הסביבה לא נטענו כלל

התסמין

ה-JWT תמיד נחתם עם סוד ברירת מחדל קשיח, ו-`AI_SERVICE_URL` תמיד היה `undefined` — כלומר כל מנגנון ה-AI היה מושבת בשקט והחזיר תמיד "לא כפילות".

הסיבה — והיא מעניינת

`app.ts` קרא ל-`dotenv.config()` בשורה 12. אבל ב-ESM כל ה-`import` מתבצעים לפני גוף המודול.
`services/jwt.ts` קרא את `process.env.JWT_SECRET` ברמת המודול — כלומר לפני ש-dotenv רץ.

התיקון: `server.ts` מייבא `loadEnv` ראשון, ואף מודול לא קורא `process.env` בזמן `import`. יש בדיקת רגרסיה שמאמתת שהטוקן לא מתאמת מול הסוד הישן.

באג 2 — ה-Error Middleware היה קוד מת

התסמין

היה handler גלובלי ב- `app.ts`, אבל הוא מעולם לא נקרא.

הסיבה

כל controller עטף את עצמו ב- `try/catch` והחזיר `res.status(500)` בעצמו, ולכן `next(err)` לא נקרא אף פעם.
התיקון: מחיקת כל ה- `try/catch` והוספת `asyncHandler`.

באג 3 — הגבלת מצב הדמו מעולם לא נתפסה

התסמין

אדמין-דמו יכול היה למחוק נתונים אמיתיים, למרות שנכתב קוד שאמור למנוע זאת.

הסיבה

`demoRestrict.ts` השווה מול `'admin-demo@cityscan.demo'` בעוד `auth.ts` יצר את החשבון בתור `'guest_admin@cityscan.com'`. שתי כתובות שונות — התנאי תמיד יצא "לא דמו" והכל עבר.
התיקון: שניהם קוראים מ- `config.demo.adminEmail`. זה בדיוק העיקרון שזהות מגיעה מקונפיגורציה ולא מליטרל שמשוכפל בין קבצים — הלקח מ- `@Value("${spring.application.name}")`.

נמצא ועדיין לא תוקן — מיועד לשלב 2

XSS מאוחסן ב- `MapComponent.tsx`: ה-`popup` נבנה עם `innerHTML` ומזריק לתוכו את תיאור המפגע — טקסט חופשי שמגיע ממשתמש, בלי `escaping`.

12. בדיקות

35 בדיקות אינטגרציה, שרצות מול MongoDB אמיתי חד-פעמי שעולה לכל ריצה. אותו עיקרון של Testcontainers ב-SmartCollect — סביבה נקייה ומבודדת שלא תלויה במחשב המפתח — בלי לדרוש התקנת Docker.

קובץ	מה מכסה
<code>hazards.test.ts</code>	חזרה ה-JSON, כפילות ברדיוס, fail-open של ה-AI, דחיסת תמונות, מחיקת קבצים, הרשאות, חזרה השגיאות
<code>auth.test.ts</code>	הרשמה, התחברות, גרסיית ה-JWT, מניעת account enumeration, הגבלת הדמו
<code>sse.test.ts</code>	זרם אירועים חי מקצה לקצה

בדיקה ששווה להראות

הבדיקה שמאמתת ששני מיילים שגויים — אחד לא קיים ואחד עם סיסמה שגויה — מחזירים **בדיוק אותה הודעה**. אחרת נקודת הקצה הופכת ל-oracle שמאפשר לגלות אילו כתובות רשומות במערכת.

13. שיקולי עלות (FinOps)

רכיב	עלות	הבחירה
מסד נתונים	0\$	MongoDB Atlas M0 — חינמי ולא נרדם
מפה	0\$	Leaflet + OSM, בלי מפתח ובלי כרטיס אשראי
אחסון תמונות	0\$	דיסק ה-VPS. דחיסה הופכת את זה לבר-קיימא
LLM	0\$	מסלול חינמי של Gemini
דומיין + SSL	0\$	DuckDNS + Let's Encrypt
שרת VPS	5\$/חודש	ההוצאה החוזרת היחידה בכל הארכיטקטורה

החישוב שמאחורי דחיסת התמונות

תמונה מהטלפון היא ~1.5MB ב-base64. אחסון בתוך מסמך MongoDB היה מתקרב לתקרת 16MB למסמך, וממלא Atlas M0 (512MB) אחרי כ-100 דיווחים בלבד. אחרי `sharp` — שינוי גודל ל-1600px והמרה ל-WebP — התמונה יורדת ל-~150KB, ודיסק של 25GB מחזיק בסדר גודל של 100,000 תמונות.

ניקוי ה-EXIF הוא לא רק חיסכון: תמונות מטלפון נושאות קואורדינטות GPS מוטמעות, והתמונות האלה מוצגות בפומבי על מפה. הדיווח כבר נושא מיקום מפורש — אין סיבה לפרסם גם את המקום המדויק שבו המדווח עמד.

שלב 2 — אופטימיזציית ה-Frontend

- תיקון ה-XSS ב- `MapComponent.tsx` — עדיפות ראשונה
- מחיקת הפוליטציה והחלפתה ב- `EventSource` על ה-SSE שכבר נבנה
- מיטוב רינדורים עם `useCallback` על ה-handlers
- אירוח עצמי של אייקוני המפה — כרגע נטענים מ-GitHub ומ-cdnjs, תלות בצד שלישי
- Clustering לסמנים, ובסיס-מפה נקי יותר (CARTO Voyager) — שיפור חינומי

שלב 3 — פריסה לשרת פרטי

- `Dockerfile` ל-Backend + `docker-compose.yml` לייצור
- n8n לאוטומציות רקע — תזכורות על מפגעים ישנים, סיכומים שבועיים, תיוג חומרה
- DuckDNS + Nginx Proxy Manager ל-HTTPS אוטומטי

נקודת החיבור ל-n8n כבר קיימת

`EventsService` הוא מנגנון פרסום. אותו אירוע שמזין את ה-SSE יכול להזין גם webhook יוצא ל-n8n — מנוי שני על אותו בוס, בלי ארכיטקטורה נוספת. השרת פולט פעם אחת, שני צרכנים.

שים לב: Nginx Proxy Manager מבצע buffering כברירת מחדל, ולכן הנתיב של ה-SSE יצטרך `proxy_buffering off` — אחרת האירועים יישבו בבאפר ולא יגיעו.

15. איך להסביר את זה לאנשים

מבנה של 90 שניות

- מה זה — "מערכת דיווח מפגעים עירוניים. תושב לוחץ על מפה, מצלם, מדווח."
- הארכיטקטורה — "שלושה שירותים: API, React ב-TypeScript/Node, ושירות AI בפייתון. ה-API בנוי בארכיטקטורת שכבות קפדנית."
- מה מעניין טכני — "מניעת כפילויות בשתי רמות: שאילתה גאוגרפית לוודאי, LLM למעורפל."
- מה למדתי — הבאגים. זה החלק שמבדיל אותך.

שאלות שישאלו, ותשובות

"למה כל כך הרבה תיקיות לפרויקט קטן?"

"כי הקונטרולר היה 500 שורות שהחזיקו שאילתות, מתמטיקה גאוגרפית, קריאות AI ובדיקות הרשאה. אי אפשר היה לבדוק שום דבר מזה בלי להרים שרת. עכשיו הלוגיקה העסקית נבדקת בלי HTTP בכלל. המבחן שאני מציע: להחליף את ספק ה-LLM זה קובץ חדש ושורה אחת ב- `container.ts`."

"מה זה ה-boundaries האלה? זה לא סתם כפילות?"

"זו הפרדה בין איך שהנתונים נשמרים לאיך נשלחים. הדוגמה הכי טובה היא המשתמש: ל- `UserBoundary` פשוט אין שדה סיסמה, אז היא לא יכולה לדלוף. הקוד הישן עשה `delete o.password` בשלושה מקומות — וזה דולף כברירת מחדל, כי שדה חדש בסכמה נשלח עד שמישהו יזכור למחוק אותו."

"למה SSE ולא WebSocket?"

"הזרימה חד-כיוונית — השרת מודיע ללקוח שנוסף מפגע. WebSocket דו-כיווני, כבד יותר בזיכרון, ודורש ספרייה. SSE זה HTTP רגיל, עובד דרך כל פרוקסי, ומתחבר מחדש לכד. על VPS של \$5 עם זיכרון מוגבל זה משנה."

"איך אתה יודע שזה עובד?"

"35 בדיקות אינטגרציה מול מסד נתונים אמיתי שעולה נקי בכל ריצה. הן מכסות את חוזה ה-JSON מול ה-Frontend, את קודי השגיאה, את ההרשאות — ויש בדיקת רגרסיה ספציפית לבאג טעינת ה-env, שמאמתת שהטוקן לא נחתם עם הסוד הישן."

אם ישאלו על חולשות — אל תתחמק

יש XSS פתוח ב-Frontend שזוהה ומתוזמן לשלב 2. ה-SSE עובד בהנחת instance יחיד; ריבוי instances היה דורש Redis pub/sub. אין עדיין pagination אמיתי — יש תקרה של 500. לומר את זה בעצמך זה סימן לבגרות הנדסית, לא לחולשה.

שלוש נקודות שכדאי להשאיר אצל המאזין

- באג `dotenv` ב-ESM — מדגים הבנה של סדר טעינת מודולים, לא רק שימוש בספרייה.
- Fail-open ב-AI — מדגים חשיבה על מצבי כשל, לא רק על המסלול המוצלח.
- ניקוי EXIF — מדגים מודעות לפרטיות שאיש לא ביקש.

